Name: Toyosi Ogundeyi
Blazer ID: Ogundeyi
Course: CS 532

Report on Producer-Consumer Communication Using Processes, Threads, Pipes, and Signals

Objective

The objective of this assignment was to implement a producer-consumer system using processes, POSIX threads, pipes, synchronization mechanisms, signals, and file operations in C. The program demonstrates communication between a parent and child process while ensuring safe synchronization among multiple producer and consumer threads.

Program Design

The program creates a pipe and then uses fork() to create a child process.

The parent process creates ten producer threads. Each producer generates 500 random integers between 0 and 1000. The generated numbers are unique within each producer thread. After generation, the producer threads write the numbers to a shared pipe. A mutex protects the pipe so that only one producer writes at a time.

Once all producer threads finish generating their numbers, the parent sends a SIGUSR1 signal to the child process.

The child waits for this signal before creating twenty consumer threads. Each consumer thread reads 250 integers from the shared pipe and calculates the sum of those integers.

After all consumer threads complete, the child calculates the average of the twenty sums and writes the result to **average.txt** using standard output redirection.

Synchronization

The program uses mutexes, semaphores, and signals to coordinate execution.

- Mutexes protect pipe read operations, pipe write operations, and console output.
- Semaphores coordinate the producer threads by allowing the parent to detect when generation is complete and then release the producers to begin writing to the pipe.
- SIGUSR1 is used by the parent process to notify the child that number generation has completed before the consumer threads begin reading from the pipe.

Testing and Results

The program was compiled using:

make

and executed using:

./producer_consumer

Testing confirmed that:

- 10 producer threads generated 500 unique numbers within each producer thread.
- 20 consumer threads each read 250 integers.
- The child process waited for the SIGUSR1 signal before starting the consumer threads.
- The average of the consumer thread sums was successfully written to **average.txt**.

A sample output was:

Average of the consumer thread sums: 124172.00

The average changes with each execution because the numbers are randomly generated.

AI Tool Usage

This project was developed with assistance from **ChatGPT (OpenAI)**, an AI tool approved by UAB.

The AI was used to:

- explain process management and POSIX thread concepts;
- assist with synchronization using mutexes, semaphores, and signals;
- explain pipe communication and standard output redirection;
- review the program for correctness and debugging.

Examples of prompts used include:

1. Prompt:
Help me design a producer-consumer program using processes, POSIX threads, pipes, mutexes, semaphores, and signals.

Result:
The AI suggested organizing the program with producer threads in the parent process and

consumer threads in the child process, with a pipe providing communication between the processes.

2. Prompt:
Explain how to synchronize multiple producer threads writing to the same pipe.

Result:
The AI explained the use of a mutex to ensure that only one producer thread writes to the shared pipe at a time and recommended synchronization to prevent race conditions.

3. Prompt:
How can the child process wait until the parent finishes generating all numbers?

Result:
The AI explained the use of SIGUSR1 to notify the child after generation is complete and suggested having the child wait for the signal before starting its consumer threads.

4. Prompt:
How can standard output be redirected to a text file in C?

Result:
The AI explained the use of freopen() to redirect standard output to average.txt before printing the calculated average.

Conclusion

This assignment provided practical experience with process creation, multithreading, synchronization, inter-process communication, signals, and file operations in C. The completed program successfully coordinated multiple producer and consumer threads, safely transferred data through a shared pipe, calculated the required average, and stored the result in an output file.